UNIVERSIDAD CHAMPAGNAT - PROGRAMACIÓN DISTRIBUIDA

Sistema Distribuido de Venta de Entradas

Microservicios, Concurrencia con Redis, Docker y Seguridad JWT

Sistema de compra de entradas diseñado en 3 microservicios independientes con Node.js, MongoDB, Redis, JWT y Docker Compose para evitar sobreventas ante múltiples pedidos simultáneos.

User Service (:3000)

Marcos

- Registro y Login seguro
- Hashing con Bcrypt
- Emisión de tokens JWT

Inventory Service (:3001)

Nahuel

- Catálogo y control de stock
- Lock distribuido en Redis
- Base propia en MongoDB

Checkout Service (:3002)

Lisandro

- Orquesta las compras
- Propaga JWT hacia Inventory
- Timeouts y pruebas E2E

Stack: Node.js 20 | Express | MongoDB 7 | Redis 7 | JWT | Docker Compose | Swagger | Winston

1. Problema que Resolvemos y Evolución (TP1 a TP7)

OBJETIVO

El Problema: Compras Simultáneas sobre Stock Limitado

- Cuando varias personas compran la última entrada al mismo tiempo, sin sincronización se produce sobreventa (stock negativo).
- Además, un sistema monolítico mezcla responsabilidades y si falla un módulo puede caer toda la aplicación.

Etapa 1: Base (TP1 - TP3)

- API REST en Node.js y Express.
- Primer Lock en Redis para evitar condiciones de carrera.
- Persistencia en MongoDB, logs con Winston y Swagger.

Etapa 2: Microservicios (TP5-6)

- División en 3 servicios independientes (User, Inventory, Checkout).
- Cada servicio corre en su contenedor Docker.
- Inventory encapsula el stock y su Lock en Redis.

Etapa 3: Seguridad (TP7)

- Contraseñas encriptadas con Bcrypt.
- Tokens JWT para autenticar sin estado.
- Permisos por rol (USER y ADMIN) y validación de propietario.

2. Arquitectura: 3 Microservicios y Bases Independientes

ARQUITECTURA

Flujo en Red Interna Docker (backend-net)

user-service (:3000)

Registro, Login y JWT

BD: MongoDB (users-db)

checkout-service (:3002)

Recibe la compra (Stateless)

Sin BD propia - Propaga JWT

PUT /reducir

---------->

inventory-service (:3001)

Stock + Lock Distribuido

Redis + MongoDB (inventory-db)

Regla Clave: Cada Servicio es Dueño de su Dato

- User Service maneja únicamente usuarios en users-db.
- Inventory Service es el único que accede a inventory-db y a Redis.
- Checkout Service no tiene base de datos: coordina el pedido y llama a Inventory por HTTP.

Ventajas del Diseño Desacoplado

- Si cae user-service, los clientes que ya tienen token JWT pueden seguir comprando normalmente.
- Checkout no necesita saber cómo se bloquea el stock: solo llama al endpoint de Inventory.
- Cada servicio se puede escalar o actualizar por separado.

3. Control de Concurrencia (Redis) y Fallos Parciales

CONCURRENCIA

Cómo Evitamos la Sobreventa en Inventory Service

1. Lock en Redis: Al llegar PUT /api/inventario/reducir, intenta crear una clave temporal por 5 segundos (SET NX PX 5000).

2. Si otro usuario ya la tiene: Responde al instante 409 Conflict sin tocar MongoDB.

3. Si obtuvo el lock: Descuenta 1 unidad en MongoDB validando stock > 0.

4. Al terminar (finally): Libera el lock con un Script Lua verificando su UUID.

Manejo de Fallos Parciales en Checkout Service

- Timeout de 3 segundos (AbortController): Si Inventory tarda o está caído, Checkout corta la espera.

- Respuesta controlada: Checkout nunca se cuelga; devuelve 503 Service Unavailable y registra el error en Winston.

- Solución ante cortes a mitad de camino: Uso de Idempotency-Key para reintentos seguros y patrón Saga para compensar.

Flujo Atómico en inventory-service/index.js

```
lock = await redisClient.set(lockKey, ownerId, { NX: true, PX: 5000 }); // Si es null -> 409 Conflict
```

```
await Inventario.findOneAndUpdate({ producto, stock: { $gt: 0 } }, { $inc: { stock: -1 } });
```

```
// Finally -> Script Lua borra el lock solo si el valor coincide con ownerId
```

4. Seguridad (JWT + RBAC) e Infraestructura Docker

SEGURIDAD & DOCKER

Seguridad Aplicada (TP7)

- Contraseñas seguras: Guardadas con hash bcrypt (10 rondas), nunca en texto plano.

- Tokens JWT (HS256): Emitidos al hacer login. Cada servicio verifica el token por su cuenta.

- Roles (RBAC):

  * Público: consultar catálogo (/api/inventario) y login/registro.

  * USER: comprar entradas y ver solo su propio perfil.

  * ADMIN: cargar o modificar stock, listar y borrar usuarios.

- Higiene del código: Sin claves ni certificados .pfx ni archivos de logs subidos a Git.

Infraestructura en Docker Compose

- 6 contenedores coordinados: 3 microservicios Node.js + 2 MongoDB + 1 Redis.

- Red privada backend-net: Solo se exponen los puertos 3000, 3001 y 3002; las bases de datos quedan ocultas.

- Volúmenes persistentes: Los datos de MongoDB no se pierden al reiniciar contenedores.

- Healthchecks: Cada servicio espera a que su base de datos y Redis estén sanos antes de arrancar.

```
docker compose up -d --build
```

5. Pruebas Automatizadas Reproducibles y Conclusiones

PRUEBAS

Qué Valida Nuestra Suite Automatizada (npm test)

1. Salud del sistema: Verifica que los 3 servicios, MongoDB y Redis respondan 200 OK.

2. Autenticación: Registra usuarios ADMIN y USER y valida la emisión de tokens JWT.

3. Concurrencia real (Promise.all): Con stock = 1, dispara 2 compras simultáneas; 1 gana (200 OK), 1 falla (409/400) y el stock queda en 0.

4. Seguridad RBAC: Verifica rechazos 401 sin token y 403 si un USER intenta cargar stock o ver otro perfil.

Comandos para Ejecutar

1. Levantar todo el entorno:

```
docker compose up -d --build
```

2. Correr las 16 pruebas de punta a punta:

```
npm test
```

Resumen Final del Proyecto

- Arquitectura limpia en 3 microservicios.
- Cero sobreventas (Lock Redis en Inventory).
- Seguridad stateless con JWT, Bcrypt y RBAC.
- 100% reproducible con Docker y tests.